

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

M VISHRUTHA (1BF24CS157)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING**

**in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by M VISHRUTHA (**1BF24CS157**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-8
2	Implement Johnson Trotter algorithm to generate permutations.	9-11
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	12-17
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	18-22
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	23-25
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	26-29
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	30-33
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	34-39
9	Implement Fractional Knapsack using Greedy technique.	40-44
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	45-47
11	Implement "N-Queens Problem" using Backtracking.	48-50

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Topological ordering of vertices in a given digraph.

CODE:

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int n;

void topologicalSort() {
    int front = 0, rear = 0;
    int count = 0;
    int i, j;
    // Calculate indegree of each vertex
    for(i = 0; i < n; i++) {
        indegree[i] = 0;
        for(j = 0; j < n; j++) {
            indegree[i] += graph[j][i];
        }
    }
    // Insert all vertices with indegree 0 into queue
    for(i = 0; i < n; i++) {
        if(indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    printf("\nTopological Ordering: ");
    while(front < rear) {
        int vertex = queue[front++];
        printf("%d ", vertex);
        count++;
    }
}
```

```

// Update indegrees of adjacent vertices
for(i = 0; i < n; i++) {
    if(graph[vertex][i] == 1) {
        indegree[i]--;

        if(indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
}

if(count != n) {
    printf("\nGraph contains a cycle. Topological sorting not possible.");
}

printf("\n");
}

int main() {
    int i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    topologicalSort();

    return 0;
}

```

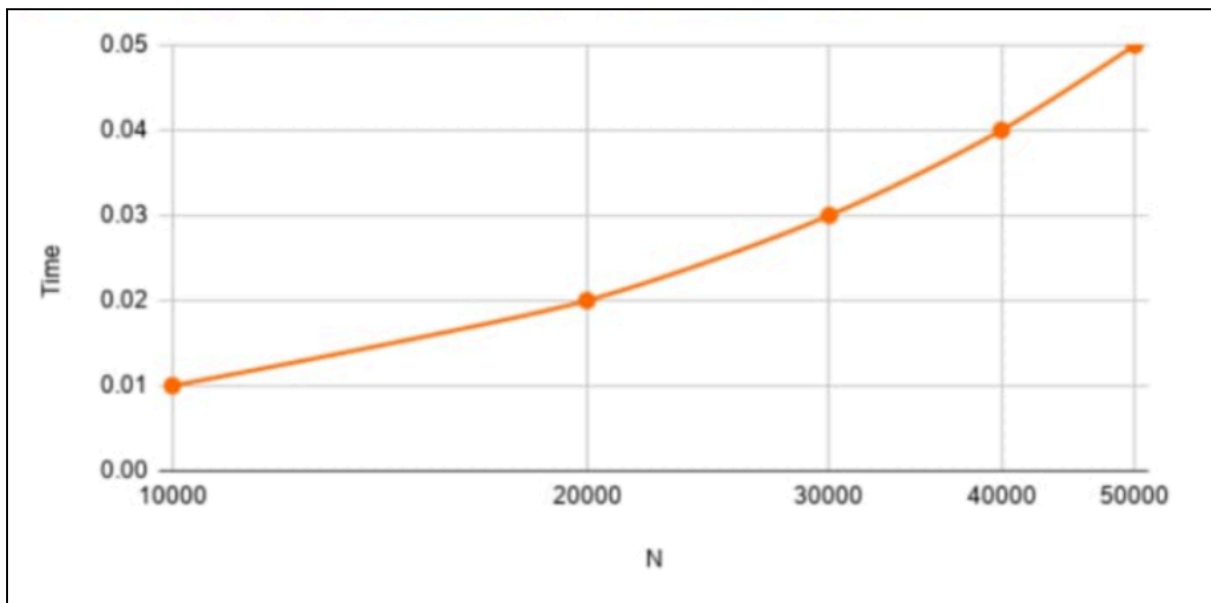
OUTPUT:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
0 0 0 1 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0

Topological Ordering: 0 1 2 3 4

Process returned 0 (0x0)   execution time : 144.090 s
Press any key to continue.
|
```

GRAPH:



LEETCODE 207- Course Schedule

CODE:

```
#include <stdbool.h>

#include <stdlib.h>

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    int indegree[2000] = {0};
    int graph[2000][2000] = {0};
    int visited[2000] = {0};
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][a] = 1;
        indegree[a]++;
    }
    int count = 0;
    while (count < numCourses) {
        int found = -1;
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0 && !visited[i]) {
                found = i;
                break;
            }
        }
        if (found == -1) {
            return false;
        }
        visited[found] = 1;
        count++;
        for (int j = 0; j < numCourses; j++) {
            if (graph[found][j]) {

```

```

        indegree[j]--;
    }
}

return true;
}

```

OUTPUT:

All Submissions

Accepted 54 / 54 testcases passed
vishrutha_03 submitted at Jun 02, 2026 22:15

Runtime
68 ms | Beats 5.17%

Memory
26.80 MB | Beats 62.57%

Code | C

207. Course Schedule

18.1K | 350

Testcase | Test Result

Accepted Runtime: 8 ms

Case 1 Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Lab program 2: Johnson Trotter algorithm to generate permutations

CODE:

```
#include <stdio.h>

#define MAX 20

#define LEFT 0
#define RIGHT 1

int findLargestMobile(int perm[], int dir[], int n) {
    int largestMobile = -1;
    int mobileIndex = -1;
    for (int i = 0; i < n; i++) {
        if (dir[i] == LEFT && i != 0 && perm[i] > perm[i - 1]) {
            if (perm[i] > largestMobile) {
                largestMobile = perm[i];
                mobileIndex = i;
            }
        }
        if (dir[i] == RIGHT && i != n - 1 && perm[i] > perm[i + 1]) {
            if (perm[i] > largestMobile) {
                largestMobile = perm[i];
                mobileIndex = i;
            }
        }
    }
    return mobileIndex;
}

void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", perm[i]);
    }
    printf("\n");
}
```

```

}

void johnsonTrotter(int n) {
    int perm[MAX], dir[MAX];
    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(perm, n);
    while (1) {
        int mobileIndex = findLargestMobile(perm, dir, n);
        if (mobileIndex == -1)
            break;
        int swapIndex;
        if (dir[mobileIndex] == LEFT)
            swapIndex = mobileIndex - 1;
        else
            swapIndex = mobileIndex + 1;
        int temp = perm[mobileIndex];
        perm[mobileIndex] = perm[swapIndex];
        perm[swapIndex] = temp;
        temp = dir[mobileIndex];
        dir[mobileIndex] = dir[swapIndex];
        dir[swapIndex] = temp;
        mobileIndex = swapIndex;
        for (int i = 0; i < n; i++) {
            if (perm[i] > perm[mobileIndex]) {
                if (dir[i] == LEFT)
                    dir[i] = RIGHT;
                else
                    dir[i] = LEFT;
            }
        }
    }
}

```

```

    }
    printPermutation(perm, n);
}
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n <= 0 || n > MAX) {
        printf("Invalid input! Enter n between 1 and %d.\n", MAX);
        return 0;
    }
    printf("\nPermutations generated using Johnson Trotter Algorithm:\n");
    johnsonTrotter(n);
    return 0;
}

```

OUTPUT:

```

Enter number of elements: 3

Permutations generated using Johnson Trotter Algorithm:
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)    execution time : 24.340 s
Press any key to continue.
|

```

Lab Program 3: Merge Sort

CODE:

```
#include<stdlib.h>

#include<stdio.h>

#include<time.h>

#include<conio.h>

void merge(int arr[], int l, int m, int r)
{
    int i=l, j=m+1, k=l;
    int temp[100000];
    while(i<=m && j<=r)
    {
        if(arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while(i<=m)
        temp[k++] = arr[i++];
    while(j<=r)
        temp[k++] = arr[j++];
    for(i=l; i<=r; i++)
        arr[i] = temp[i];
}

void mergesort(int arr[], int l, int r)
{
    if(l < r)
    {
        int m = (l+r)/2;
```

```

        mergesort(arr,l,m);
        mergesort(arr,m+1,r);
        merge(arr,l,m,r);
    }
}

int main()
{
    int n,i,arr[100000];
    clock_t start,end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        arr[i] = rand()%1000;
    printf("Generated random numbers:\n");
    for(i=0;i<n;i++)
        printf("%d ",arr[i]);
    start = clock();
    mergesort(arr,0,n-1);
    end = clock();
    time_taken = ((double)(end-start))/CLOCKS_PER_SEC;
    printf("\n\nSorted array:\n");
    for(i=0;i<n;i++)
        printf("%d ",arr[i]);
    printf("\n\nTime Taken= %f seconds\n",time_taken);
    return 0;
}

```

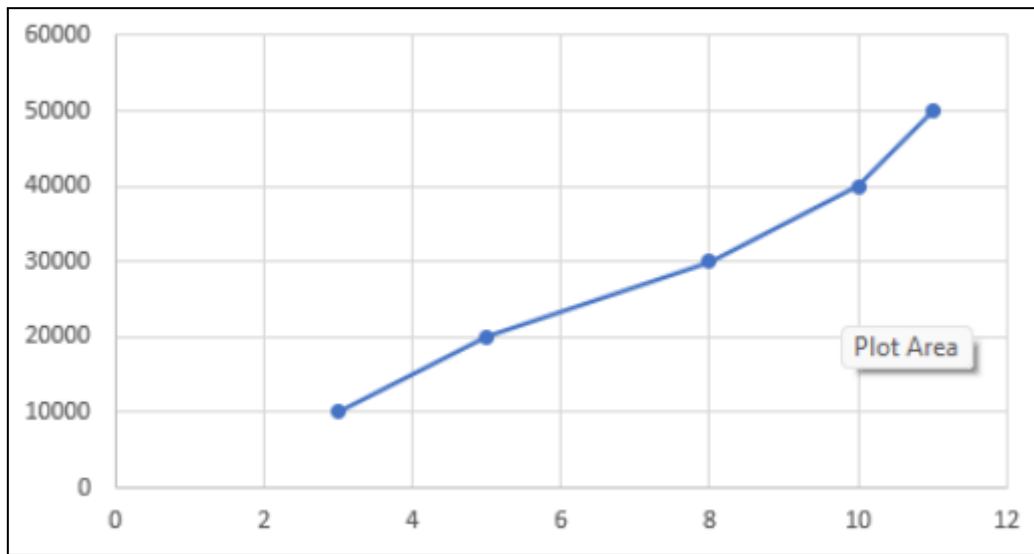
OUTPUT:

```
Enter number of elements: 50
Generated random numbers:
41 467 334 500 169 724 478 358 962 464 705 145 281 827 961 491 995 942 827 436 391 604 902 153 292 382 421 716 718 895 4
47 726 771 538 869 912 667 299 35 894 703 811 322 333 673 664 141 711 253 868

Sorted array:
35 41 141 145 153 169 253 281 292 299 322 333 334 358 382 391 421 436 447 464 467 478 491 500 538 604 664 667 673 703 70
5 711 716 718 724 726 771 811 827 827 868 869 894 895 902 912 942 961 962 995

Time Taken= 0.002000 seconds
```

GRAPH:



LEETCODE 315- Count of Smaller Numbers after self

CODE:

```
#include <stdlib.h>

typedef struct {
    int val;
    int idx;
} Pair;

void merge(Pair* arr, int left, int mid, int right,
           int* counts, Pair* temp) {
    int i = left;
    int j = mid + 1;
    int k = left;
    int rightCount = 0;
    while (i <= mid && j <= right) {
        if (arr[j].val < arr[i].val) {
            temp[k++] = arr[j++];
            rightCount++;
        } else {
            counts[arr[i].idx] += rightCount;
            temp[k++] = arr[i++];
        }
    }
    while (i <= mid) {
        counts[arr[i].idx] += rightCount;
        temp[k++] = arr[i++];
    }
    while (j <= right) {
        temp[k++] = arr[j++];
    }
}
```

```

        for (i = left; i <= right; i++) {
            arr[i] = temp[i];
        }
    }

void mergeSort(Pair* arr, int left, int right,
               int* counts, Pair* temp) {
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid, counts, temp);
    mergeSort(arr, mid + 1, right, counts, temp);
    merge(arr, left, mid, right, counts, temp);
}

int* countSmaller(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    int* counts = calloc(numsSize, sizeof(int));
    Pair* arr = malloc(numsSize * sizeof(Pair));
    Pair* temp = malloc(numsSize * sizeof(Pair));
    for (int i = 0; i < numsSize; i++) {
        arr[i].val = nums[i];
        arr[i].idx = i;
    }
    mergeSort(arr, 0, numsSize - 1, counts, temp);
    free(arr);
    free(temp);
    return counts;
}

```


OUTPUT:

← All Submissions

Accepted

66 / 66 testcases passed

v...

submitted at May 31, 2026 17:57

Analysis

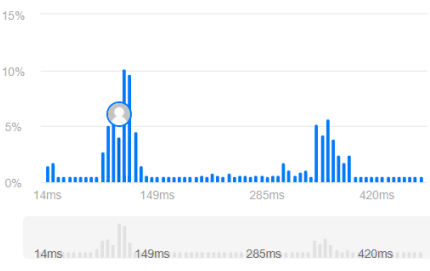
Solution

Runtime

103 ms | Beats 80.49%

Memory

75.78 MB | Beats 67.87%



The graphs show runtime and memory usage over time. The runtime graph has a peak at 149ms, and the memory graph has a peak at 149ms.

Code | C

C

Auto

1 /**

2 * Note: The returned array must be malloced, assume caller calls free().

3 */

4 /**

Saved

Testcase

Test Result

Editorial

Solutions

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

[5, 2, 6, 1]

Output

[2, 1, 1, 0]

Expected

[2, 1, 1, 0]

Lab Program 4- Quick Sort

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

// Function to swap two elements

void swap(int* a, int* b) {

    int temp = *a;

    *a = *b;

    *b = temp;

}

// Function to partition the array around a pivot

int partition(int arr[], int low, int high) {

    int pivot = arr[high]; // Choose the last element as the pivot

    int i = (low - 1); // Index of smaller element

    for (int j = low; j < high; j++) {

        // If current element is smaller than or equal to pivot

        if (arr[j] <= pivot) {

            i++;

            swap(&arr[i], &arr[j]);

        }

    }

    swap(&arr[i + 1], &arr[high]);

    return (i + 1);

}

// Function to implement Quick Sort

void quickSort(int arr[], int low, int high) {

    if (low < high) {

        // pi is partitioning index, arr[pi] is now at correct position

        int pi = partition(arr, low, high);
```

```

        // Recursively sort elements before partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

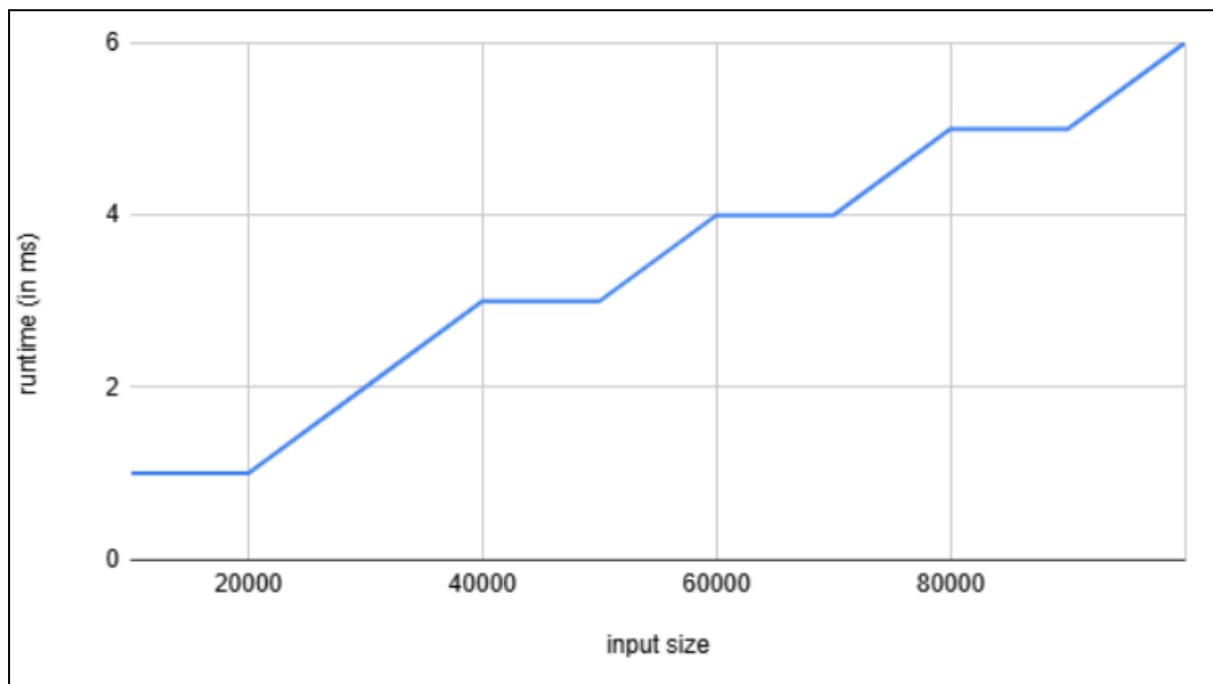
// Main program to test the Quick Sort and measure time
int main()
{
    int n, i;
    int arr[100000];
    clock_t start, end;
    double time_ms;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    /* Generate random array */
    for (i = 0; i < n; i++)
        arr[i] = rand() % 100;
    printf("\nUnsorted Array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    start = clock();
    quickSort(arr, 0, n-1);
    end = clock();
    time_ms = (((double)(end-start))/CLOCKS_PER_SEC) * 1000;
    printf("\n\nSorted array:\n");
    for(i=0;i<n;i++)
        printf("%d ",arr[i]);
    printf("\n\nTime Taken= %f milliseconds\n",time_ms);
    return 0;
}

```

OUTPUT:

```
Enter number of elements: 6  
  
Unsorted Array:  
41 67 34 0 69 24  
  
Sorted array:  
0 24 34 41 67 69  
  
Time Taken= 0.000000 milliseconds
```

GRAPH:



LEETCODE 912- Sort an Array

CODE:

```
#include <stdlib.h>

int* sortArray(int* nums, int numsSize, int* returnSize) {
    int offset = 50000;
    int range = 100001;
    int *count = calloc(range, sizeof(int));
    for (int i = 0; i < numsSize; i++) {
        count[nums[i] + offset]++;
    }
    int k = 0;
    for (int i = 0; i < range; i++) {
        while (count[i] > 0) {
            nums[k++] = i - offset;
            count[i]--;
        }
    }
    free(count);
    *returnSize = numsSize;
    return nums;
}
```

OUTPUT:

← All Submissions

Accepted 21 / 21 testcases passed

vishrutha_03 submitted at Jun 02, 2026 23:30

Runtime

15 ms | Beats 97.65%

Memory

62.18 MB | Beats 41.74%



Code | C

C Auto

```
1 #include <stdlib.h>
2
3 int* sortArray(int* nums, int numsSize, int* returnSize) {
4     int offset = 50000;
5     int range = 100001;
```

Saved

Testcase | Test Result | Editorial | Solutions

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[5,2,3,1]

Output

[1,2,3,5]

Expected

[1,2,3,5]

Lab Program 5: Heap Sort

CODE:

```
#include <stdio.h>

#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}
```

```

    }
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d integer elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start = clock();
    heapSort(arr, n);
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nSorted elements:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n\nTime taken = %lf seconds\n", time_taken);

    return 0;
}

```

OUTPUT:

```

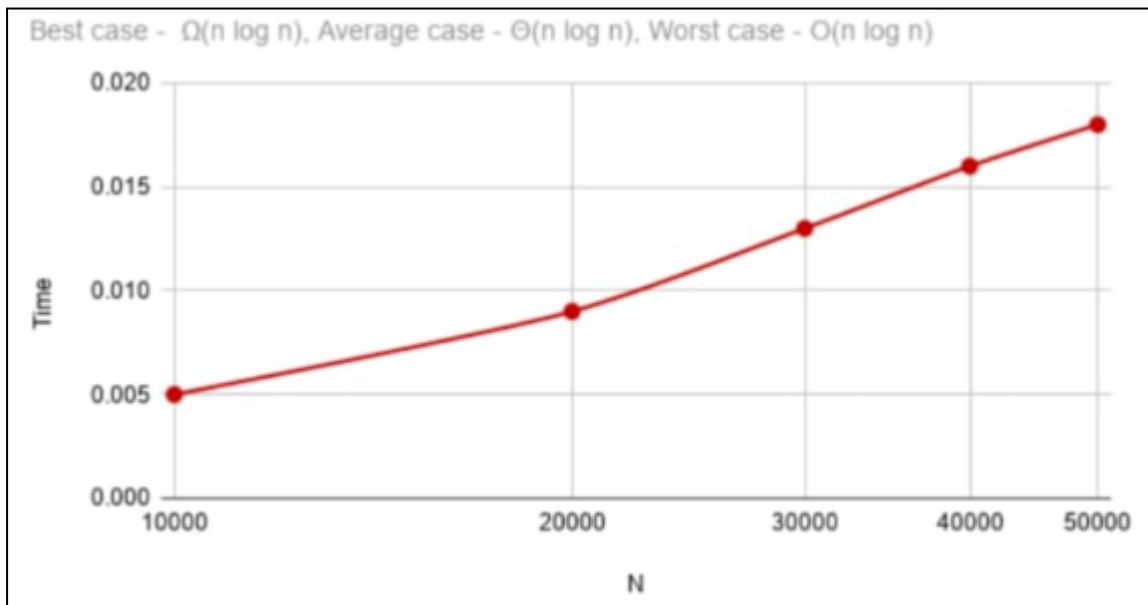
Enter number of elements: 5
Enter 5 integer elements:
20 10 50 70 30

Sorted elements:
10 20 30 50 70

Time taken = 0.000000 seconds

```


GRAPH:



Lab Program 6: 0/1 Knapsack problem using dynamic programming

CODE:

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int wt[n], val[n];
    printf("Enter weights:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);
    printf("Enter values:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);
    printf("Enter capacity: ");
    scanf("%d", &W);
    int dp[n+1][W+1];
    // Build DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i-1] <= w)
                dp[i][w] = max(
                    val[i-1] + dp[i-1][w - wt[i-1]],
                    dp[i-1][w]
                );
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    printf("Maximum value: %d", dp[n][W]);
}
```

```
        else
            dp[i][w] = dp[i-1][w];
        }
    }
    printf("\nMaximum Value = %d\n", dp[n][W]);
    return 0;
}
```

OUTPUT:

```
Enter number of items: 5
Enter weights:
10 3 6 8 1
Enter values:
1 2 4 3 1
Enter capacity: 7

Maximum Value = 5
```

LEETCODE 474- Ones and Zeroes

CODE:

```
int findMaxForm(char **strs, int strsSize, int m, int n) {  
    int dp[101][101] = {0};  
    for(int k = 0; k < strsSize; k++) {  
        int zeros = 0, ones = 0;  
        for(int i = 0; strs[k][i] != '\0'; i++) {  
            if(strs[k][i] == '0')  
                zeros++;  
            else  
                ones++;  
        }  
        for(int i = m; i >= zeros; i--) {  
            for(int j = n; j >= ones; j--) {  
                if(dp[i - zeros][j - ones] + 1 > dp[i][j])  
                    dp[i][j] = dp[i - zeros][j - ones] + 1;  
            }  
        }  
    }  
    return dp[m][n];  
}
```

OUTPUT:

Accepted 77 / 77 testcases passed

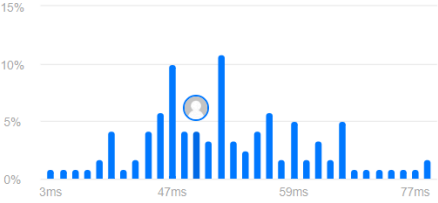
vishrutha_03 submitted at Jun 02, 2026 23:50

Runtime

49 ms | Beats 64.17%

Memory

9.98 MB | Beats 31.67%



Code | C

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

strs = ["10", "0001", "111001", "1", "0"]

m = 5

n = 3

Output

4

Expected

4

Contribute a testcase

Lab Program 7: All Pair Shortest paths problem using Floyd's algorithm

CODE:

```
#include<stdio.h>

#include<limits.h>

void main(){
    int n;

    printf("Enter the number of vertices:");

    scanf("%d",&n);

    int a[n][n];

    printf("Enter the weights and 0 for no edges:\n");

    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            {
                scanf("%d",&a[i][j]);
            }
        }
    }

    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(i!=j && a[i][j]==0) a[i][j]=INT_MAX;
        }
    }

    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            for(int k=0;k<n;k++){
                if(a[i][k]!=INT_MAX && a[k][j]!=INT_MAX){
                    if(a[i][j]>a[i][k]+a[k][j]) a[i][j]=a[i][k]+a[k][j];
                }
            }
        }
    }
}
```

```

printf("Shortest pairs:\n");
for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        if(a[i][j]==INT_MAX) printf("Infinity ");
        else printf("%d ",a[i][j]);
    }
    printf("\n");
}
}

```

OUTPUT:

```

Enter the number of vertices:4
Enter the weights and 0 for no edges:
0 5 0 10
0 0 3 0
0 0 0 1
0 0 0 0
Shortest pairs:
0 5 8 9
Infinity 0 3 4
Infinity Infinity 0 1
Infinity Infinity Infinity 0

```

LEETCODE 821- Shortest Distance to a Character

CODE:

```
#include <stdlib.h>

#include <string.h>

int* shortestToChar(char* s, char c, int* returnSize) {
    int n = strlen(s);
    *returnSize = n;
    int* ans = (int*)malloc(n * sizeof(int));
    int prev = -n;
    // Left to right
    for(int i = 0; i < n; i++) {
        if(s[i] == c)
            prev = i;
        ans[i] = i - prev;
    }
    prev = 2 * n;
    // Right to left
    for(int i = n - 1; i >= 0; i--) {
        if(s[i] == c)
            prev = i;
        int dist = prev - i;
        if(dist < ans[i])
            ans[i] = dist;
    }
    return ans;
}
```


OUTPUT:

Accepted 76 / 76 testcases passed

vishrutha_03 submitted at Jun 03, 2026 00:00

Runtime

0 ms | Beats 100.00%

Memory

11.75 MB | Beats 23.30%



Time Interval	Memory Usage (%)
0ms	100%
1ms	~1%
2ms	~1%
3ms	~1%
4ms	~1%

Code | C

23 |

```
for(int i = n - 1; i >= 0; i--) {
```

Saved

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

s =

"loveleetcode"

c =

"e"

Output

[3,2,1,0,1,0,0,1,2,2,1,0]

Expected

[3,2,1,0,1,0,0,1,2,2,1,0]

Lab program 8:

a) Prim's algorithm

CODE:

```
#include <stdio.h>

#define MAX 10

#define INF 999

int main(){

    int cost[MAX][MAX];

    int visited[MAX] = {0};

    int n, i, j;

    int min, a, b;

    int edges = 0, minCost = 0;

    printf("Enter number of vertices: ");

    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for(i = 0; i < n; i++){

        for(j = 0; j < n; j++){

            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)

                cost[i][j] = INF;

        }

    }

    visited[0] = 1;

    printf("Edges in MST:\n");

    while(edges < n - 1){

        min = INF;

        for(i = 0; i < n; i++){

            if(visited[i]){

                for(j = 0; j < n; j++){

                    if(!visited[j] && cost[i][j] < min){
```

```

        min = cost[i][j];
        a = i;
        b = j;
    }
}
}
}
printf("%d - %d = %d\n", a, b, min);
minCost += min;
visited[b] = 1;
edges++;
}
printf("Minimum Cost = %d\n", minCost);
return 0;
}

```

OUTPUT:

```

Enter number of vertices: 5
Enter adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Edges in MST:
0 - 1 = 2
1 - 2 = 3
1 - 4 = 5
0 - 3 = 6
Minimum Cost = 16

```

b) Kruskal's Algorithm

CODE:

```
#include <stdio.h>

#define MAX 20

// Structure to store edges
struct Edge
{
    int src, dest, weight;
};

// Structure for Disjoint Set
struct Subset
{
    int parent;
};

// Find function
int find(struct Subset subsets[], int i)
{
    while (subsets[i].parent != i)
    {
        i = subsets[i].parent;
    }
    return i;
}

// Union function
void Union(struct Subset subsets[], int x, int y)
{
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    subsets[xroot].parent = yroot;
}
```

```

// Sort edges by weight
void sortEdges(struct Edge edges[], int e)
{
    struct Edge temp;
    for (int i = 0; i < e - 1; i++)
    {
        for (int j = 0; j < e - i - 1; j++)
        {
            if (edges[j].weight > edges[j + 1].weight)
            {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

// Kruskal's Algorithm
void kruskalMST(struct Edge edges[], int V, int E)
{
    struct Edge result[MAX];
    struct Subset subsets[MAX];

    int e = 0;
    int i = 0;
    int totalCost = 0;

    // Initialize subsets
    for (int v = 0; v < V; v++)
    {
        subsets[v].parent = v;
    }
}

```

```

// Sort edges
sortEdges(edges, E);

// Process edges
while (e < V - 1 && i < E)
{
    struct Edge nextEdge = edges[i++];
    int x = find(subsets, nextEdge.src);
    int y = find(subsets, nextEdge.dest);
    // Include edge if it does not form a cycle
    if (x != y)
    {
        result[e++] = nextEdge;
        totalCost += nextEdge.weight;
        Union(subsets, x, y);
    }
}

// Print MST
printf("\nEdges in Minimum Spanning Tree:\n");
for (i = 0; i < e; i++)
{
    printf("%d -- %d == %d\n",
        result[i].src,
        result[i].dest,
        result[i].weight);
}

printf("\nMinimum Cost = %d\n", totalCost);
}

int main()
{
    int V, E;

```

```

struct Edge edges[MAX];

printf("Enter number of vertices: ");
scanf("%d", &V);

printf("Enter number of edges: ");
scanf("%d", &E);

printf("Enter source destination weight for each edge:\n");
for (int i = 0; i < E; i++)
{
    scanf("%d %d %d",
        &edges[i].src,
        &edges[i].dest,
        &edges[i].weight);
}

printf("\nKruskal's Algorithm - Minimum Cost Spanning Tree\n");
kruskalMST(edges, V, E);

return 0;
}

```

OUTPUT:

```

Enter number of vertices: 5
Enter number of edges: 7
Enter source destination weight for each edge:
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 3

Kruskal's Algorithm - Minimum Cost Spanning Tree

Edges in Minimum Spanning Tree:
0 -- 1 == 2
1 -- 2 == 3
3 -- 4 == 3
1 -- 4 == 5

Minimum Cost = 13

```

Lab program 9: Fractional Knapsack using Greedy Algorithm

CODE:

```
#include <stdio.h>

// Structure for items
struct Item {
    int weight;
    int profit;
    float ratio;
};

// Function to swap two items
void swap(struct Item *a, struct Item *b) {
    struct Item temp = *a;
    *a = *b;
    *b = temp;
}

// Sort items based on profit/weight ratio (Descending) - Bubble Sort
void sortItems(struct Item arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j].ratio < arr[j+1].ratio) {
                swap(&arr[j], &arr[j+1]);
            }
        }
    }
}

// Fractional Knapsack function
float fractionalKnapsack(struct Item arr[], int n, int capacity) {
    float totalProfit = 0.0;
    for (int i = 0; i < n; i++) {
        if (capacity >= arr[i].weight) {
```



```

        // Take full item

        totalProfit += arr[i].profit;

        capacity -= arr[i].weight;
    } else {

        // Take fraction of item

        totalProfit += arr[i].ratio * capacity;

        break;

    }

}

return totalProfit;
}

int main() {

    int n, capacity;

    printf("Enter number of items: ");

    scanf("%d", &n);

    struct Item arr[n];

    // Input weights and profits

    for (int i = 0; i < n; i++) {

        printf("Enter weight and profit of item %d: ", i+1);

        scanf("%d %d", &arr[i].weight, &arr[i].profit);

        arr[i].ratio = (float)arr[i].profit / arr[i].weight;

    }

    printf("Enter capacity of knapsack: ");

    scanf("%d", &capacity);

    // Sort items by ratio

    sortItems(arr, n);

    float maxProfit = fractionalKnapsack(arr, n, capacity);

    printf("Maximum profit = %.2f\n", maxProfit);

    return 0;

}

```

OUTPUT:

```
Enter number of items: 5
Enter weight and profit of item 1: 1 2
Enter weight and profit of item 2: 2 4
Enter weight and profit of item 3: 5 2
Enter weight and profit of item 4: 9 1
Enter weight and profit of item 5: 9 9
Enter capacity of knapsack: 10
Maximum profit = 13.00
```

LEETCODE 435- Non Overlapping Intervals

CODE:

```
int compare(const void* a, const void* b) {
    int* x = *(int**)a;
    int* y = *(int**)b;
    return x[1] - y[1];
}

int eraseOverlapIntervals(int** intervals, int intervalsSize, int* intervalsColSize) {
    if(intervalsSize == 0)
        return 0;
    qsort(intervals, intervalsSize,
        sizeof(int*), compare);
    int count = 0;
    int prevEnd = intervals[0][1];
    for(int i = 1; i < intervalsSize; i++) {
        if(intervals[i][0] < prevEnd) {
            count++;
        }
        else {
            prevEnd = intervals[i][1];
        }
    }
    return count;
}
```

OUTPUT:

Accepted 60 / 60 testcases passed

Submitted at May 28, 2026 22:17

Analysis

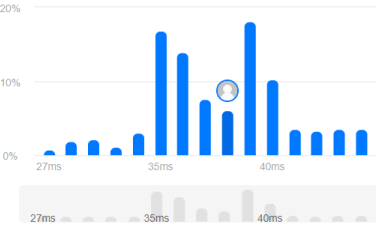
Solution

Runtime

38 ms Beats 53.16%

Memory

35.76 MB Beats 13.51%



Code | C

1 int compare(const void* a, const void* b) {

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

intervals =

[[1,2], [2,3], [3,4], [1,3]]

Output

1

Expected

1

Contribute a testcase

Lab Program 10: Shortest paths to other vertices using Dijkstra's algorithm

CODE:

```
#include <stdio.h>

#define MAX 10

#define INF 9999

int graph[MAX][MAX];
int visited[MAX];
int dist[MAX];

void dijkstra(int n, int source) {
    int i, j, u, min;
    for(i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[source] = 0;
    for(i = 0; i < n - 1; i++) {
        min = INF;
        u = -1;
        for(j = 0; j < n; j++) {
            if(!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }
        visited[u] = 1;
        for(j = 0; j < n; j++) {
            if(!visited[j] &&
                graph[u][j] != 0 &&
                dist[u] + graph[u][j] < dist[j]) {
```

```

        dist[j] = dist[u] + graph[u][j];
    }
}
}

printf("\nShortest distances from vertex %d:\n", source);
for(i = 0; i < n; i++) {
    printf("To %d = %d\n", i, dist[i]);
}
}

int main() {
    int n, i, j, source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter weighted adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    dijkstra(n, source);

    return 0;
}

```

OUTPUT:

```
Enter number of vertices: 4
Enter weighted adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Enter source vertex: 0

Shortest distances from vertex 0:
To 0 = 0
To 1 = 2
To 2 = 5
To 3 = 6
```

Lab Program 11: “N-Queens Problem” using Backtracking

CODE:

```
#include <stdio.h>

#define MAX 20

int board[MAX][MAX];

int solutionCount = 0;

int isSafe(int row, int col, int n) {
    int i, j;
    for (i = 0; i < row; i++) {
        if (board[i][col] == 1)
            return 0;
    }
    for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1)
            return 0;
    }
    for (i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (board[i][j] == 1)
            return 0;
    }
    return 1;
}

void printBoard(int n) {
    printf("\nSolution %d:\n", ++solutionCount);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j] == 1)
                printf("Q ");
            else
                printf(". ");
        }
    }
}
```



```

    }

    printf("\n");
}

}

void solveNQueens(int row, int n) {
    if (row == n) {
        printBoard(n);
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isSafe(row, col, n)) {
            board[row][col] = 1;
            solveNQueens(row + 1, n);
            board[row][col] = 0; // backtrack
        }
    }
}

int main() {
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    if (n <= 0 || n > MAX) {
        printf("Invalid input!\n");
        return 0;
    }
    solveNQueens(0, n);
    if (solutionCount == 0)
        printf("\nNo solution exists.\n");
    else
        printf("\nTotal Solutions = %d\n", solutionCount);
}

```

```
    return 0;  
}
```

OUTPUT:

```
Enter number of queens: 4
```

```
Solution 1:
```

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

```
Solution 2:
```

```
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

```
Total Solutions = 2
```